

Proyecto Mentora

Tutorial: Hola Mundo con Django + PostgreSQL + Docker

Universidad Nacional de Colombia — Sede Bogotá
Facultad de Ingeniería — Departamento de Sistemas e Industrial
Curso: Ingeniería de Software 1 (2016701)

Entrega 04 — Módulo de Clean Code

Integrantes: Pablo Rosales Obando, Gabriel Bustos Moreno, Alexander Mosquera Ramirez, Diego Esteban Jimenez Gutierrez

Lenguaje: Python 3.11 | Framework: Django 4.2 | ORM: Django ORM

1. Introducción

Este tutorial explica paso a paso cómo construir una aplicación backend funcional desde cero usando Python con Django, conectada a una base de datos PostgreSQL, todo orquestado con Docker Compose. El objetivo es que cualquier persona pueda replicar el proceso siguiendo los pasos en orden.

Se aplicaron principios de Clean Code a lo largo de todo el proyecto: nombres descriptivos, separación de responsabilidades, configuración por variables de entorno y estructura modular por aplicaciones.

2. Requisitos Previos

Antes de comenzar se necesita tener instalado Docker Desktop en el equipo. Todo lo demás (Python, Django, PostgreSQL) corre dentro de los contenedores, por lo que no hace falta instalarlo en la máquina local.

2.1 Instalar Docker Desktop

En Windows

1. Descargar Docker Desktop desde <https://www.docker.com/products/docker-desktop/>
2. Ejecutar el instalador y reiniciar el equipo cuando lo pida.
3. Abrir Docker Desktop y esperar a que el icono de la ballena quede en verde (Engine running).
4. Abrir PowerShell y verificar que Docker funciona correctamente:

NOTA: Todos los comandos de este tutorial se ejecutan en PowerShell de Windows, dentro de la carpeta del proyecto. Nosotros usamos el docker.

```
# Ejecutar en PowerShell
# Verificar que Docker está instalado correctamente
docker --version

# Verificar que Docker Compose está disponible
docker compose version
```

Si ambos comandos muestran una versión, Docker está listo para usar.

En Mac

```
# Ejecutar en Terminal
brew install --cask docker
```

En Linux (Ubuntu)

```
# Ejecutar en Terminal
sudo apt update
sudo apt install docker.io docker-compose -y
sudo usermod -aG docker $USER
# Cerrar sesión y volver a entrar para aplicar el grupo
```

3. Stack Tecnológico

Estas son las tecnologías usadas en el proyecto y el rol que cumple cada una:

Componente	Tecnología	Versión	Rol en el proyecto
Lenguaje	Python	3.11	Lenguaje principal del backend
Framework web	Django	4.2.11	Estructura MVC, ORM, admin, migraciones
ORM	Django ORM	incluido	Convierte clases Python en tablas SQL
API REST	Django REST Framework	3.15.1	Serializa modelos a JSON y gestiona vistas HTTP
Base de datos	PostgreSQL	15	Motor de base de datos relacional
Contenedores	Docker + Compose	v2	Levanta y conecta los servicios (web + BD)
Configuración	python-decouple	3.8	Lee variables de entorno, evita hardcodear credenciales
Conector BD	psycopg2-binary	2.9.9	Permite que Python se conecte a PostgreSQL

4. Estructura del Proyecto

El proyecto sigue una arquitectura modular. Cada aplicación tipo django tiene su propia responsabilidad y sus propios archivos de modelos, vistas, serializadores y rutas. Este es un principio clave de Clean Code: una sola responsabilidad por módulo.

```
Mentora/
├── manage.py           # Punto de entrada de Django (no modificar)
├── Dockerfile          # Define como construir la imagen del servidor web
├── docker-compose.yml  # Orquesta los servicios: BD + servidor web
├── requirements.txt    # Lista de dependencias Python del proyecto
├── setup.sh            # Script de automatización para Linux/Mac
├── setup.bat           # Script de automatización para Windows
└── config/             # Configuración global del proyecto Django
```

└─ __init__.py	# Marca la carpeta como paquete Python
└─ settings.py	# Ajustes: BD, apps instaladas, zona horaria, etc.
└─ urls.py	# Rutas principales que conectan a cada app
└─ wsgi.py	# Punto de entrada para el servidor web
└─ usuarios/	# Módulo que gestiona usuarios y roles
└─ models.py	# Define las entidades Usuario y Rol en la BD
└─ views.py	# Controladores: reciben petición y devuelve respuesta
└─ serializers.py	# Convierte objetos Python a JSON y viceversa
└─ urls.py	# Rutas específicas de este módulo
└─ tutorias/	# Módulo de tutorías (expandible)
└─ urls.py	# Rutas del modulo (vacío por ahora)

5. Archivos de Configuración

5.1 requirements.txt

Lista todas las dependencias Python. Docker las instala automáticamente al construir la imagen:

```
# Contenido del archivo requirements.txt
Django==4.2.11
psycopg2-binary==2.9.9
djangorestframework==3.15.1
python-decouple==3.8
```

5.2 Dockerfile

Le dice a Docker cómo construir la imagen del servidor web. Se parte de una imagen base de Python 3.11, se instalan las dependencias del sistema necesarias para conectarse a PostgreSQL, luego las dependencias Python, y finalmente se copian los archivos del proyecto:

```
# Contenido del archivo Dockerfile
FROM python:3.11-slim

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Instala gcc y libpq-dev: necesarios para compilar psycopg2 (conector Python-PostgreSQL)
RUN apt-get update && apt-get install -y gcc libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Copia e instala dependencias Python (se hace antes que el código para aprovechar cache)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copia todos los archivos del proyecto al contenedor
COPY . .

# Garantiza que los __init__.py existen (necesarios para que Python encuentre los módulos)
RUN touch config/__init__.py usuarios/__init__.py tutorias/__init__.py

# Puerto que expone el servidor de Django
EXPOSE 8000
```

```
# Comando que se ejecuta al iniciar el contenedor
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

5.3 docker-compose.yml

Orquesta los dos servicios del proyecto: la base de datos (db) y el servidor web (web). El servicio web espera a que la BD esté completamente lista (healthcheck) antes de iniciarse, evitando errores de conexión:

```
# Contenido del archivo docker-compose.yml
version: '3.9'

services:

  # Servicio 1: Base de datos PostgreSQL
  db:
    image: postgres:15-alpine          # Imagen oficial de PostgreSQL
    container_name: mentora_db
    restart: unless-stopped
    environment:
      POSTGRES_DB: mentora_db          # Nombre de la base de datos
      POSTGRES_USER: mentora_user      # Usuario de la BD
      POSTGRES_PASSWORD: mentora_pass  # Contraseña de la BD
    ports:
      - "5432:5432"                   # Expone el puerto de PostgreSQL al host
    volumes:
      - postgres_data:/var/lib/postgresql/data # Persiste los datos entre reinicios
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mentora_user -d mentora_db"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Servicio 2: Servidor Django
  web:
    build: .                          # Construye la imagen usando el Dockerfile local
    container_name: mentora_web
    restart: unless-stopped
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./app                          # Monta el código local dentro del contenedor
    ports:
      - "8000:8000"                     # Expone el servidor Django al host
    depends_on:
      db:
        condition: service_healthy     # Solo inicia cuando la BD esté lista
    environment:
      DB_NAME: mentora_db
      DB_USER: mentora_user
      DB_PASSWORD: mentora_pass
      DB_HOST: db                       # 'db' es el nombre del servicio en la red Docker
      DB_PORT: "5432"

volumes:
  postgres_data:                      # Volumen que persiste los datos de PostgreSQL
```

6. Configuración Django (config/settings.py)

En vez de escribir las credenciales de la BD directamente en el código (lo que sería un problema de seguridad y de Clean Code), se usan variables de entorno leídas con python-decouple. Los valores vienen del bloque 'environment' del docker-compose.yml:

```
# Contenido relevante de config/settings.py
import os
from decouple import config

# Clave secreta de Django (nunca compartir en produccion)
SECRET_KEY = config("SECRET_KEY", default="django-insecure-clave-desarrollo")
DEBUG = True
ALLOWED_HOSTS = ["*"]

# Aplicaciones instaladas en el proyecto
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "rest_framework", # Django REST Framework
    "usuarios", # Modulo de usuarios del proyecto
    "tutorias", # Modulo de tutorias del proyecto
]

ROOT_URLCONF = "config.urls" # Archivo de rutas principal

# Conexion a la base de datos PostgreSQL
# Los valores se leen desde las variables de entorno del docker-compose.yml
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": config("DB_NAME", default="mentora_db"),
        "USER": config("DB_USER", default="mentora_user"),
        "PASSWORD": config("DB_PASSWORD", default="mentora_pass"),
        "HOST": config("DB_HOST", default="db"),
        "PORT": config("DB_PORT", default="5432"),
    }
}

LANGUAGE_CODE = "es-co"
TIME_ZONE = "America/Bogota"
```

7. Entidades del Proyecto (usuarios/models.py)

Los modelos son clases Python que Django ORM convierte automáticamente en tablas de la base de datos. Cada campo de la clase se convierte en una columna de la tabla. Se aplica Clean Code con docstrings en cada clase y métodos con nombres que expresan claramente su intención:

```
# Contenido de usuarios/models.py
from django.db import models

class Rol(models.Model):
    """Representa un rol dentro del sistema (ej: tutor, estudiante, profesor)."""
```

```

nombre = models.CharField(max_length=50)

class Meta:
    db_table = "rol" # Nombre exacto de la tabla en la BD

def __str__(self):
    return self.nombre

class Usuario(models.Model):
    """Usuario del sistema Mentora. Puede tener uno o mas roles."""

    nombre = models.CharField(max_length=100, null=True, blank=True)
    apellido = models.CharField(max_length=100)
    correo = models.CharField(max_length=150)
    password_hash = models.CharField(max_length=255)
    activo = models.BooleanField(default=True)
    creado_en = models.DateTimeField(auto_now_add=True) # Se llena automaticamente

    class Meta:
        db_table = "usuario" # Nombre exacto de la tabla en la BD

    def __str__(self):
        return f"{self.nombre} {self.apellido} <{self.correo}>"

    def nombre_completo(self) -> str:
        """Retorna el nombre completo del usuario."""
        return f"{self.nombre} {self.apellido}"

    def esta_activo(self) -> bool:
        """Verifica si el usuario esta habilitado en el sistema."""
        return self.activo

```

8. Endpoint Hola Mundo (usuarios/views.py)

La vista HolaMundoView es el endpoint principal del tutorial. Cuando recibe una petición GET, consulta la base de datos contando cuántos usuarios y roles existen, y devuelve esa información en formato JSON. Esto demuestra que la conexión a la BD está activa y las entidades están correctamente instanciadas:

```

# Contenido de usuarios/views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .models import Usuario, Rol

class HolaMundoView(APIView):
    """
    Endpoint de prueba que verifica la conexion a la base de datos
    y la instanciacion minima de las entidades del sistema.
    """

    def get(self, request):
        # Consulta cuantos registros existen en cada tabla
        total_usuarios = Usuario.objects.count()
        total_roles = Rol.objects.count()

        # Construye la respuesta con informacion del sistema
        respuesta = {

```

```

        "mensaje": "Hola Mundo desde Mentora API",
        "framework": "Django 4.2 + Django REST Framework",
        "base_de_datos": "PostgreSQL 15 (conexion activa)",
        "entidades_registradas": {
            "usuarios": total_usuarios,
            "roles": total_rols,
        },
        "estado": "OK",
    }
    return Response(respuesta, status=status.HTTP_200_OK)

```

Rutas del módulo (usuarios/urls.py)

El archivo de rutas conecta las URLs con las vistas correspondientes:

```

# Contenido de usuarios/urls.py
from django.urls import path
from .views import HolaMundoView, UsuarioListView, UsuarioDetailView

urlpatterns = [
    path("hola-mundo/", HolaMundoView.as_view(), name="hola-mundo"),
    path("", UsuarioListView.as_view(), name="usuario-list"),
    path("<int:usuario_id>/", UsuarioDetailView.as_view(), name="usuario-detail"),
]

```

Rutas principales (config/urls.py)

El archivo de rutas principal del proyecto conecta cada prefijo de URL con el módulo correspondiente:

```

# Contenido de config/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/usuarios/", include("usuarios.urls")), # Todas las rutas de usuarios
    path("api/tutorias/", include("tutorias.urls")), # Todas las rutas de tutorias
]

```

Nota: Para simplificar el tutorial, únicamente se desarrollará el endpoint hola-mundo/.

9. Pasos para Ejecutar el Proyecto

NOTA: Todos los comandos que siguen se ejecutan en PowerShell de Windows (o Terminal en Mac/Linux), nosotros usamos la de terminal de docker dentro de la carpeta raíz del proyecto (Mentora/).

Paso 9.1 — Ir a la carpeta del proyecto

Abrir PowerShell y navegar a la carpeta donde está el proyecto:

```
# Ejecutar en PowerShell – ajustar la ruta según tu equipo
cd C:\Users\TuUsuario\Documents\Mentora
```

Paso 9.2 — Construir y levantar los contenedores

Este comando lee el docker-compose.yml, construye la imagen de Django usando el Dockerfile, descarga la imagen de PostgreSQL, y levanta ambos servicios conectados entre sí. La primera vez tarda unos minutos porque descarga las imágenes base:

```
# Ejecutar en PowerShell – esperar hasta ver 'Watching for file changes'
docker compose up --build
```

Cuando el terminal muestre el mensaje 'Watching for file changes with StatReloader', el servidor está corriendo. Para continuar con los siguientes pasos, abrir una segunda ventana de PowerShell sin cerrar esta.

NOTA: El flag --build le dice a Docker que reconstruya la imagen aunque ya exista. Esto es necesario cuando se cambia el código o el Dockerfile. En las ejecuciones siguientes se puede usar solo 'docker compose up -d' para levantar en segundo plano.

Paso 9.3 — Aplicar las migraciones

Django necesita crear las tablas en la base de datos. Este comando lee los modelos del proyecto y genera las tablas correspondientes en PostgreSQL. Ejecutar en una segunda ventana de PowerShell:

```
# Ejecutar en una segunda ventana de PowerShell
docker compose exec web python manage.py migrate
```

Este comando le dice a Docker que ejecute 'python manage.py migrate' dentro del contenedor 'web'. Si todo va bien, se verá una lista de migraciones aplicadas con 'OK' al lado de cada una.

Paso 9.4 — Verificar que los contenedores están activos

Para confirmar que ambos servicios están corriendo correctamente:

```
# Ejecutar en PowerShell
docker ps -a
```

Se deben ver dos contenedores: mentora_db con estado 'Up (healthy)' y mentora_web con estado 'Up'. Si alguno aparece como 'Exited', revisar los logs con 'docker compose logs web'.

Paso 9.5 — Probar el endpoint en el navegador

Con los contenedores activos, abrir el navegador y acceder a la siguiente URL:

```
# Pegar en la barra de direcciones del navegador
http://localhost:8000/api/usuarios/hola-mundo/
```

El navegador debe mostrar la respuesta JSON del servidor. Esto confirma que Django esta corriendo, la conexión a PostgreSQL funciona, y las entidades están correctamente instanciadas.

10. Prueba desde Postman

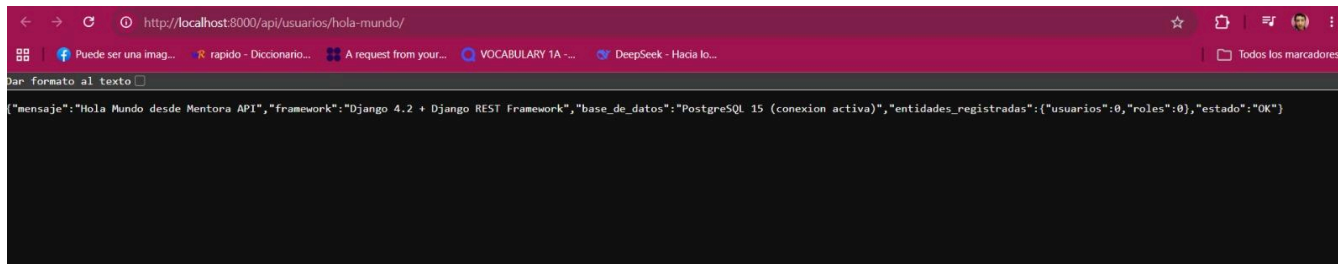
Además del navegador, se puede usar Postman para hacer la petición y ver la respuesta de forma más clara. Configurar la petición de la siguiente forma:

Campo	Valor
Método HTTP	GET
URL	http://localhost:8000/api/usuarios/hola-mundo/
Headers	(no se requiere ninguno)
Body	(no se requiere, es GET)
Autorización	(ninguna, el endpoint es público)

La respuesta esperada con código HTTP 200 OK es la siguiente:

```
# Respuesta JSON del servidor (HTTP 200 OK)
{
  "mensaje": "Hola Mundo desde Mentora API",
  "framework": "Django 4.2 + Django REST Framework",
  "base_de_datos": "PostgreSQL 15 (conexion activa)",
  "entidades_registradas": {
    "usuarios": 0,
    "roles": 0
  },
  "estado": "OK"
}
```

Esta respuesta confirma tres cosas fundamentales de la entrega: (1) la conexión a PostgreSQL está activa, (2) las entidades Usuario y Rol están correctamente instanciadas y se pueden consultar, y (3) la API REST con Django REST Framework está funcionando correctamente.



11. Comandos Utiles en PowerShell

Referencia rápida de los comandos más usados durante el desarrollo. Todos se ejecutan desde la carpeta raíz del proyecto:

Comando	Que hace
<code>docker compose up --build</code>	Construye la imagen y levanta todos los servicios
<code>docker compose up -d</code>	Levanta los servicios en segundo plano (sin bloquear la terminal)
<code>docker compose down</code>	Detiene y elimina los contenedores (los datos persisten en el volumen)
<code>docker compose logs web</code>	Muestra los logs del servidor Django
<code>docker compose logs web -f</code>	Muestra los logs en tiempo real (Ctrl+C para salir)
<code>docker compose ps</code>	Muestra el estado de todos los contenedores
<code>docker ps -a</code>	Muestra todos los contenedores del sistema con su estado
<code>docker compose exec web python manage.py migrate</code>	Aplica las migraciones de la BD
<code>docker compose exec web python manage.py makemigrations</code>	Genera nuevas migraciones al cambiar modelos
<code>docker compose exec web python manage.py createsuperuser</code>	Crea un usuario administrador para el panel /admin/
<code>docker compose restart web</code>	Reinicia solo el servidor Django (util al cambiar codigo)

12. Principios de Clean Code Aplicados

A continuación se muestra como se aplicó cada principio de Clean Code en el proyecto:

Principio	Como se aplica en Mentora
Nombres descriptivos	nombre_completo(), esta_activo(), HolaMundoView — cada nombre dice exactamente que hace
Una responsabilidad por módulo	Cada app (usuarios, tutorías) maneja un solo dominio del negocio
No hardcodear credenciales	python-decouple lee DB_NAME, DB_USER, DB_PASSWORD del entorno, nunca del código
Docstrings en clases y métodos	Todas las clases y métodos tienen descripción en triple comilla
Separación config/código	settings.py solo tiene configuracion, ninguna logica de negocio
Estructura modular	Cada app tiene sus propios models, views, serializers y urls
Variables con intención clara	activo, creado_en, password_hash — sin abreviaciones ni nombres genéricos
Comentarios que explican el por qué	Los comentarios en el código explican la razón, no el que (que ya lo dice el código)